

Flash Sale Platform

Data Models & Schemas

Generated: 2026-09-26 20:32

Architecture

Redis: Inventory, holds, queue (fast, temporary). **PostgreSQL:** Orders, payments, saga state (durable, system of record). **Kafka:** Event log for durability and replay.

Redis Keys (with {eventId} hash tag)

Key	Type	Contents	TTL
ev:{eventId}:avail	String (int)	Tickets available	None
ev:{eventId}:sold	String (int)	Tickets confirmed sold	None
ev:{eventId}:total	String (int)	Total tickets	None
ev:{eventId}:holds	Sorted Set	reservationId → expiresAt (ms)	None
ev:{eventId}:user:{userId}	String (int)	Qty held/bought by user	Sale end
ev:{eventId}:res:{reservationId}	Hash	userId, qty, status, expiresAt	~24h after final
ev:{eventId}:stream	Stream	Outbox of reservation events	Trimmed ~MAXLEN
ev:{eventId}:queue	Sorted Set	queueTicket → arrival ms	Sale end

Invariant: avail + held + sold = total. held = SUM(qty) from holds ZSET. Consider caching as ev:{eventId}:held counter.

PostgreSQL Schema (Prisma)

Event

```
id (String, PK)
title, totalTickets, priceInPaise, perUserLimit
saleStartsAt, saleEndsAt
createdAt
```

Order

```
id (String, PK = reservationId)
eventId (FK), userId, qty, amountInPaise
state (HELD, PAYMENT_PENDING, PAID, CONFIRMED, PAYMENT_FAILED, EXPIRED, RELEASED, REFUND_PENDING, REFUND_FAILED)
holdExpiresAt, version
Indexes: (eventId, state), (userId), (state, holdExpiresAt)
```

Payment

```
id (String, PK)
orderId (FK), idempotencyKey (UNIQUE)
gatewayRef (UNIQUE, nullable), status (INITIATED, SUCCEEDED, FAILED, REFUNDED)
amountInPaise, createdAt
```

OutboxMessage

```
id (String, PK)
topic, key, payload (JSON)
createdAt, publishedAt (nullable)
Index: (publishedAt, createdAt)
```

ProcessedEvent

```
eventId (String, PK)
consumer, processedAt
```

IdempotencyRecord

```
key (String, PK)
requestHash, response (JSON), createdAt
```

Kafka Topics & Events

Event Envelope (all messages)

```
{
  "eventId": "01JABCD...",
  "type": "reservation.created",
  "schemaVersion": 1,
  "occurredAt": "2026-10-12T10:00:01.123Z",
  "aggregateId": "res_01JABCE...",
  "payload": { ... }
}
```

Trace context travels in Kafka headers, not body. Use ULIDs for IDs (time-sortable).

Event Types v1

Type	Topic	Key	Payload Fields	Producer → Consumer
reservation.created	reservations.events	reservationId	eventId, userId, qty, expiresAt	Relay → Order
reservation.released	reservations.events	reservationId	reason (expired, payment failed)	Relay → Order
payment.succeeded	payments.events	reservationId	paymentId, amount, gatewayRef	Payment → Order
payment.failed	payments.events	reservationId	paymentId, reason	Payment → Order
order.confirmed	orders.events	reservationId	orderId, qty	Order → Ticket
refund.issued	payments.events	reservationId	paymentId, amount	Payment → Order

Key by reservationId: One reservation's events live on one partition, in order. Spreads one hot event (many people buying same ticket) across partitions.

Design Decisions

Order.id = reservationId

Natural idempotency: redelivered 'reservation.created' hits primary-key conflict, no duplicate order.

Money as Paise (integers)

Never floats. Avoids rounding errors and simplifies auditing.

No Per-Ticket Rows (General Admission)

Inventory is a counter, not a seat bitmap. Simpler, faster. Assigned seating is a stretch goal.

Guarded State Transitions

UPDATE ... SET state = \$new WHERE id = \$id AND state = \$expected. Retries are safe.

Redis + Postgres, not Redis alone

Redis: fast inventory checks. Postgres: durability, saga state, audit trail.